Lab Handout 2-H

Resource Utilization, Timing Analysis, and Bitstream Sign-Off

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

1 Objective

Close Module 2 by running the full `ingest_top` design through Vivado implementation, analyzing the resource and timing reports, fixing any remaining negative slack, and producing a signed-off bitstream ready for Module 3 system integration.

2 Prerequisites

- Lab Handout 2-G (end-to-end simulation passing all three scenarios).

3 Procedure

3.1 Step 1 — Run implementation

1. Flow Navigator → **Run Implementation**. Wait for completion.
2. If synthesis or implementation errors appear, fix them before proceeding. Warnings about unused signals are acceptable; warnings about inferred latches are not.
3. Open the **Utilization** report. Record LUT, FF, Block Random Access Memory (BRAM), and Digital Signal Processing (DSP) usage.

Expected utilization envelope

For the reference implementation on the Artix-7 100T, expect roughly: LUT 1–3 %, FF 0.5–2 %, BRAM 2 tiles, DSP 0. Numbers substantially higher than this usually point at an accidental block-RAM inference or a combinational feedback loop.

3.2 Step 2 — Timing report

1. Open **Report Timing Summary**. Worst Negative Slack (WNS) must be ≥ 0 for both setup and hold.
2. If setup slack is negative, pull up the top 10 failing paths. Typical culprits: the Cyclic Redundancy Check (CRC) XOR tree (see 2-E), a long combinational compare in the parser Finite State Machine (FSM), or an unregistered output into `led`.
3. Register any failing output. Re-run implementation and confirm positive slack.

[Figure placeholder]

`LH2H_timing_summary.png`

Vivado Timing Summary report with all four categories (WNS/TNS, WHS/THS, WPWS/TPWS) showing positive slack.

Figure 1: Vivado Timing Summary report with all four categories (WNS/TNS, WHS/THS, WPWS/TPWS) showing positive slack.

3.3 Step 3 — Clock domain crossings

Verify that Vivado's **Report CDC** shows only the expected crossing: `uart_rxd` into `sysclk` via `sync_2ff`. If any unexpected crossings appear, trace them before proceeding.

3.4 Step 4 — Generate the bitstream

1. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` must appear in **bold**. If anything else is bold (a testbench from a recent simulation run, for example), right-click `ingest_top` → **Set as Top** before generating the bitstream.

2. Click **Generate Bitstream**. The output is `ingest_top.bit`.

3. Open the **Power** report. Dynamic power should be under 200 mW for this design.

3.5 Step 5 — Hardware sanity check

1. Program the Nexys A7 from Hardware Manager.

2. Connect the STM32 interceptor Universal Asynchronous Receiver/Transmitter (UART) `TX` to the Pmod JA pin declared in `ingest_top.xdc`.

3. Power on the cooperative-target pair (Target A and Target B) plus the interceptor (Threat). Per the `ingest_top` LED map established in LH2-F: `LED(0)` (UART byte strobe) flickers rapidly, `LED(1)` (`frame_valid`) pulses at the per-frame cadence, `LED(2)` (First In, First Out (FIFO) empty) is on most of the time, `LED(3)` (FIFO full) stays dark, `LED(4)` (`frame_reject`) stays dark on a clean stream, and `LED(5)` (`feat_busy`) flickers briefly during each feature update. `LED(15:8)` shows the selected feature byte (driven by `SW(7:0)` = row, `SW(12:8)` = byte index).

4. Add an Integrated Logic Analyzer (ILA) probe on `frame_valid` and `frame_reject`. Confirm that `frame_valid` pulses at roughly the Module 1 ping-pong cadence (DATA → ACK pairs) and `frame_reject` is rare.

[Figure placeholder]

`LH2H_ila_capture.png`

ILA capture on hardware: `frame_valid` pulses at the cooperative-target ping-pong cadence while `frame_reject` stays quiet.

Figure 2: ILA capture on hardware: `frame_valid` pulses at the cooperative-target ping-pong cadence while `frame_reject` stays quiet.

3.6 Step 6 — Export reports to machine-readable format

The Module 5 system specification table (and the Module 2 resource row in the final report) requires exact LUT/FF/BRAM/WNS numbers. Export them from Vivado's Tcl console now so they are available without re-opening the GUI later.

```
# Open the implemented design if not already open
open_run impl_1

# Export utilization to CSV
report_utilization -hierarchical -file impl/util.rpt

# Export timing summary
report_timing_summary -file impl/timing.rpt -warn_on_violation

# Export power
report_power -file impl/power.rpt
```

Listing 1: Run in the Vivado Tcl Console after implementation completes.

Then run the following Python snippet once to extract the numbers into a structured `impl_summary.json` that the Module 5 spec-table script can read directly:

```
import json, re
from pathlib import Path

def parse_util(rpt: str) -> dict:
    patterns = {
        "lut":  r"Slice LUTs\s*\|\s*(\d+)",
        "ff":   r"Slice Registers\s*\|\s*(\d+)",
        "bram": r"Block RAM Tile\s*\|\s*(\d+)",
        "dsp":  r"DSPs\s*\|\s*(\d+)",
    }
    result = {}
    for key, pat in patterns.items():
        m = re.search(pat, rpt)
        result[key] = int(m.group(1)) if m else None
```

```python
    return result

def parse_timing(rpt: str) -> dict:
    m = re.search(r"WNS\(ns\)\s+TNS\(ns\).*?\n\s*([-\d.]+)\s+([-\d.]+)",
        rpt, re.S)
    return {"wns_ns": float(m.group(1)) if m else None,
            "tns_ns": float(m.group(2)) if m else None}

util = parse_util(Path("util.rpt").read_text(errors="replace"))
timing = parse_timing(Path("timing.rpt").read_text(errors="replace"))

summary = {"module": "Module2_ingest_top", **util, **timing}
Path("impl_summary.json").write_text(json.dumps(summary, indent=2))
print(json.dumps(summary, indent=2))
```

Listing 2: `parse_vivado_reports.py` — run from the `impl/` directory.

Example output:

```{
  "module": "Module2_ingest_top",
  "lut": 312,
  "ff": 198,
  "bram": 2,
  "dsp": 0,
  "wns_ns": 2.14,
  "tns_ns": 0.0
}
```

Commit `impl_summary.json` alongside the bitstream. This file is the authoritative source for Table 2 in the final report.

3.7 Step 7 — Archive the sign-off artifacts

Collect into a single directory:

- `ingest_top.bit`.
- `impl_summary.json` (from Step 6).
- The raw `util.rpt`, `timing.rpt`, and `power.rpt` files.
- The Module 2-G regression results (diff files from all three scenarios).
- A one-page sign-off summary confirming LUT/FF/BRAM within the expected envelope, WNS ≥ 0, and dynamic power < 200 mW.

4 Verification Checklist

- ☐ Implementation completes with zero errors and no inferred-latch warnings.
- ☐ WNS and WHS both ≥ 0.
- ☐ Report CDC shows only the `uart_rxd` → `sysclk` crossing.

- ☐ Dynamic power $< 200\,\text{mW}$.
- ☐ Hardware ILA capture confirms `frame_valid` activity on a live cooperative-target ping-pong stream (with PAUSE bursts visible when the Threat user button is held).

Common Pitfalls

- **Negative hold slack**: almost always a missing register between two ultra-short paths. Add a single flip-flop rather than trying to slow the clock.
- **Inferred latch warning**: an incomplete `if`/`case` in a combinational process. Provide default assignments at the top of every combinational block.
- **Unexpected BRAM inference**: the parser's 32-byte `payload` register array accidentally sized above Vivado's distributed-RAM threshold. Force distributed RAM with the `(*ram_style = "distributed"*)` attribute.

Lab Handout 2-H Deliverable

1. `ingest_top.bit` and raw post-implementation reports (`util.rpt`, `timing.rpt`, `power.rpt`).
2. `impl_summary.json` with LUT/FF/BRAM/WNS fields populated (this feeds Table 2 of the final report directly).
3. One-page sign-off summary confirming all fields within spec.
4. ILA screenshot from live hardware.

Glossary

BRAM	Block Random Access Memory	1, 2, 3
CRC	Cyclic Redundancy Check	1
DSP	Digital Signal Processing	1
FIFO	First In, First Out	2
FPGA	Field-Programmable Gate Array	1
FSM	Finite State Machine	1
ILA	Integrated Logic Analyzer	2, 3
RTL	Register-Transfer Level	1
UART	Universal Asynchronous Receiver/Transmitter	2